CS3230 Design and Analysis of Algorithms	October 9, 2019

All-Pairs Shortest Paths

Name: Divesh Aggarwal

Problem 1-1 (Predecessor)

(a) Describe a modification of LeyzorekAPSP that returns an array of predecessor pointers, in addition to the array of shortest path distances, still in $O(V^3 \log V)$ time.

Solution:

Let $pred[u, v]$ denote the predecessor of vertex u on a shortest path starting from vertex u. Recall that in initialization, we do not use any middle vertices, thus we initialize it to Null if there is it is the predecessor of itself or if there is no edge from u to v; and to u if there is an edge from u to v.

If the algorithm relax an edge $u \to v$ using a middle vertex x, we set the predecessor of v by the predecessor of v on the current shortest path starting from vertex x found so far.

```
LeyzorekAPSP(V, E, w):
    for all vertices u
        for all vertices v
            dist[u, v] ← w(u → v)
            if u = v OR w(u → v) = ∞
                pred[u, v] ← Null
            else
                pred[u, v] ← u
    for i ← 1 to ⌈lg V⌉
        for all vertices u
            for all vertices v
                for all vertices x
                    if dist[u, v] > dist[u, x] + dist[x, v]
                        dist[u, v] ← dist[u, x] + dist[x, v]
                        pred[u, v] ← pred[x, v]
```

Clearly it still runs in $O(V^3 \log V)$.

□

(b) Describe a modification of FloydWarshall that returns an array of predecessor pointers, in addition to the array of shortest path distances, still in $O(V^3)$ time.

Solution:

Let $pred[u, v]$ denote the predecessor of vertex u on a shortest path starting from vertex u. Recall that in initialization, we do not use any intermediate vertices, thus we initialize it to NULL if there is it is the predecessor of itself or if there is no edge from u to v; and to u if there is an edge from u to v.

If the algorithm relax an edge $u \to v$ using an intermediate vertex r, we set the predecessor of v by the predecessor of v on the current shortest path starting from vertex r found so far.

```
FloydWarshall(V, E, w):
    for all vertices u
        for all vertices v
            dist[u, v] ← w(u → v)
            if u = v OR w(u → v) = ∞
                pred[u, v] ← Null
            else
                pred[u, v] ← u
    for all vertices r
        for all vertices u
            for all vertices v
                if dist[u, v] > dist[u, r] + dist[r, v]
                    dist[u, v] ← dist[u, r] + dist[r, v]
                    pred[u, v] ← pred[r, v]
```

Clearly it still runs in $O(V^3)$.

□

Problem 1-2 (Arbitrage Cycle)

In this problem we will discover how you, yes **you**, can be employed by Wall Street and cause a major economic collapse! The arbitrage business is a money-making scheme that takes advantage of differences in currency exchange. In particular, suppose 1 US dollar buys 120 Japanese yen, 1 yen buys 0.01 euros, and 1 euro buys 1.2 US dollars. Then, a trader starting with \$1 can convert their money from dollars to yen, then from yen to euros, and finally from euros back to dollars, ending with \$1.44! The cycle of currencies \$ → ¥ → € → \$ is called an **arbitrage cycle**. Of course, finding and exploiting arbitrage cycles before the prices are corrected requires extremely fast algorithms.

Suppose n different currencies are traded in your currency market. You are given the matrix $Exch[1..n, 1..n]$ of exchange rates between every pair of currencies; for each i and j, one unit of currency i can be traded for $Exch[i, j]$ units of currency j. (Do not assume that $Exch[i, j] \cdot Exch[j, i] = 1$.)

(a) Describe an algorithm that returns an array $MaxAmt[1..n]$, where $MaxAmt[i]$ is the maximum amount of currency i that you can obtain by trading, starting with one unit of currency 1, assuming there are no arbitrage cycles.

Solution:

We convert the problem of finding the maximum amount of currency obtained by trading starting from one unit of currency 1 to finding the shortest paths starting from vertex 1 as follows. We create a complete graph where $V = \{1, \ldots, n\}$ and weight $w(i \to j) = -\log Exch[i, j]$ for all $i, j \in V$. Notice that by the monotonicity property of logarithms and $\log a \cdot b = \log a + \log b$, we have that a shortest path from 1 to u of length ℓ correspond to the largest sequence of trades that starts with one unit of currency 1 and ends with $10^{-\ell}$ units of currency u, and vice versa.

Since the resulting weights may be negative and by the assumption that there are no arbitrage cycles which means there are no negative cycles, we can use BellmanFord to compute the shortest paths from vertex 1 to every other vertex. Let its output be $dist[1\,..\,n]$. We compute and output $MaxAmt[1\,..\,n]$ where $MaxAmt[i] = 10^{-i}$.

□

(b) Describe an algorithm to determine whether the given matrix of currency exchange rates creates an arbitrage cycle.

Solution:

We convert the problem as in (a) and run one more inner iteration of BellmanFord to detect negative cycles. Report an arbitrage cycle detected iff a negative cycle detected.

□

(c) Modify your algorithm from part (b) to actually return an arbitrage cycle, if it exists.

Solution:

We convert the problem as in (b) and reduce it to return a negative cycle. We use and modify the version of BellmanFord that maintains an array of pointers to the predecessor $pred[\,]$. After running one more inner iteration of BellmanFord and detecting that a negative cycle exits through $u \to v$ that is changed (relaxed), we first find a node in a negative cycle that either includes that edge or its predecessor as follows.

```
BFFindVertexNegCycle(V, E, w):
    ...
    for all edges u → v
        if dist[v] > dist[u] + w(u → v)
            pred[v] ← u
            while u is not marked
                mark u
                u ← pred[u]
            return u
```

Once we found that vertex u, we can find and return the cycle by following the predecessor pointers until we encounter marked u again.

□

Problem 1-3 (Another APSP Algorithm)

Let $G = (V, E)$ be a directed graph with weighted edges; edge weights could be positive, negative, or zero.

(a) How would we delete an arbitrary vertex v from this graph, without changing the shortest-path distance between any other pair of vertices? Describe an algorithm that constructs a directed graph $G' = (V \setminus \{v\}, E')$ with weighted edges, such that the shortest-path distance between any two vertices in G' is equal to the shortest-path distance between the same two vertices in G, in $O(V^2)$ time.

Solution:

We construct the graph $G' = (V', E', w')$ from $G = (V, E, w)$ that maintains the shortest-path distance between any other pair of vertices as follows. We remove any path $u \to v \to x$ and add an edge $u \to x$ if it does not already exist, and set its the weight to be the length of the path if it is smaller than the previous weight. Wlog v is the last vertex V. Thus, we modify the weight matrix of G to get w' in $O(V^2)$ time as follows.

```
DeleteLastVertex(w, w'):
    for u ← 1 to V−1
        for x ← 1 to V−1
            w'[u, x] ← min{w[u, x], w[u, V] + w[V, x]}
```

Consider an arbitrary path $p \in G$. Let $w(p)$ be the length of p in G and $w'(p)$ be the length of p in G'. Let p' be the p with $u \to v \to x$ replaced by $u \to x$ if it exists, $p' = p$ otherwise. Since we set $w'[u, x] \leftarrow \min\{w[u, x],\ w[u, v] + w[v, x]\}$, we get that $w'(p') = \min\{w(p), w(p')\}$.

This implies that every path in G' has the same length as some path in G with the same endpoints. Thus, the shortest-path distances in G' cannot be smaller than the shortest-path distances in G. Also, if p is a shortest path in G, then $w(p) \leq w(p')$, so $w'(p') = w(p)$. Thus, the shortest-path distances in G' cannot be larger than the shortest-path distances in G. Therefore, the shortest-path distances in G and G' are equal.

□

(b) Now suppose we have already computed all shortest-path distances in G'. Describe an algorithm to compute the shortest-path distances in the original graph G from v to every other vertex, and from every other vertex to v, all in $O(V^2)$ time.

Solution:

Notice that for all vertices $u \neq v$, the shortest-path distances in the original graph G from v to every other vertex u must satisfy

$$dist[u, v] = \min\{\, dist[u, x] + w[x, v],\ \text{for all } x \neq v \,\}$$

$$dist[v, u] = \min\{\, w[v, z] + dist[z, u],\ \text{for all } z \neq v \,\}$$

where by assumption all $dist[u, x]$ and $dist[z, u]$ have already been computed. Wlog v is the last vertex V. Thus, we have the following algorithm to compute those shortest-path distances that runs in $O(V^2)$ time.

```
RestoreLastVertex(w, dist):
    for u ← 1 to V−1
        dist[u, V] ← ∞
        for x ← 1 to V−1
            dist[u, V] ← min{ dist[u, V], dist[u, x] + w[x, V] }
        dist[V, u] ← ∞
        for z ← 1 to V−1
            dist[V, u] ← min{ dist[V, u], w[V, z] + dist[z, u] }
```

□

(c) Combine parts (a) and (b) into another all-pairs shortest path algorithm that runs in $O(V^3)$ time. (The resulting algorithm is *almost* the same as Floyd-Warshall!)

Solution:

Combining both parts where we "delete" the vertices as in (a) to compute the shortest-path distances and then "restore" the vertices as in (b) using the computed shortest-path distances, we have the following algorithm that runs in $O(V^3)$ time.

```
AnotherAPSP(w, dist):
    for i ← 1 to V
        for j ← 1 to V
            dist[i, j] ← w[i, j]
    for i ← V down to 1
        for j ← 1 to i−1
            for k ← 1 to i−1
                dist[j, k] ← min{ dist[j, k], dist[j, i] + dist[i, k] }
    for i ← 1 to V
        for j ← 1 to i−1
            for k ← 1 to i−1
                dist[i, j] ← min{ dist[i, j], dist[i, k] + dist[k, j] }
                dist[j, i] ← min{ dist[j, i], dist[j, k] + dist[k, i] }
```

□

Problem 1-4 (Avoiding Tolls)

You are given a map $G = (V, E)$ with cities V connected by roads E. Each road (edge) is labeled with a weight which is a real number. You are located in city $s \in V$. You are also given an array A of boolean values that tells you if there is a toll on that road. More precisely, for road $e \in E$ we have $A[e] = 1$ if and only if there is a toll on e. Being the low budget traveler that you are, your budget allows for at most one such toll to be payed for any given trip (path).

Let $d(s, t)$ denote the minimum possible sum of weights of a path from s to t for any $s, t \in V$. If there is no path from s to t, then $d(s, t) = \infty$, and if there is a path from s to t that contains a negative cycle, then $d(s, t) = -\infty$ (since one can cycle along the path an arbitrary number of times).

Similarly, let $c(s, t)$ denote the minimum possible sum of weights of a path from s to t that passes through at most 1 toll.

(a) Construct a graph $G' = (V', E')$ and mappings $f : V \mapsto V'$, $g : V \mapsto V'$ such that $|V'| = 2|V|$, $|E'| \leq 2|E| + |V|$ and for any $s, t \in V$, $c(s, t) = d(f(s), g(t))$. Namely, you reduce the "constrained" problem on G to "unconstrained" problem in G'. Remember to consider the case when $c(s, t)$ is $-\infty$, and prove the correctness of your solution.

Solution: Let $V' = V_1 \cup V_2$, where V_1 and V_2 are two disjoint copies of V. For any $v \in V$, we denote the corresponding vertices in V_1 and V_2 as v_1 and v_2, respectively.

The edges in E' with corresponding edge weights is defined as follows. For all $v \in V$, we add an edge in E' with weight 0 from v_1 to v_2. In addition, for all $e = (u, v) \in E$,

- Add two edges (u_1, v_1) and (u_2, v_2) to E' with edge weight $W[u, v]$ if and only if $A[e] = 0$, i.e., if e is not a toll edge.
- Add an edge (u_1, v_2) to E' with edge weight $W[u, v]$ if and only if $A[e] = 1$, i.e., if e is a toll edge.

Thus, the number of edges in E' is at most $2|E| + |V|$. Furthermore, we define $f(v) = v_1$, and $g(v) = v_2$. Then for any path from vertex u to v in G that passes through 0 or 1 toll, there is a corresponding path in G' from u_1 to v_2 that has the same total weight. Also, for any path from u_1 to v_2 in G', there is a corresponding path from u to v in G that passes through at most one toll and has the same total weight. □

(b) Give an algorithm that takes as input s and finds a shortest path with at most one toll road from s to all cities in V. Analyze the running time of your algorithm.

Solution: Run Bellman-Ford Algorithm on G'. The running time is $O(nm + n^2)$. □

(c) Now assume your buddy Billybob who works at a major airlines company has given you a free plane ticket to any city in V, meaning you can start your trip at any node. As before, once you start your road trip, you are still refusing to pay more then a single toll on any such trip. To help plan the trip, your job is to give an algorithm to find a shortest path with at most one toll road between *all pairs* of cities in V. Analyze the running time of your algorithm. (**Hint**: Remember Johnson.)

Solution: Run Johnson's Algorithm on G'. The running time is $O(nm + n^2 \log n)$. $\square$

(d) Here you will solve the problem in part (c) directly on graph G, without constructing the helper graph G'. Let $W = \{w(i,j)\}$ be the original edge weight matrix and $W' = \{w'(i,j)\}$ be the same edge matrix except we replace $w'(i,j) = \infty$ if $A(i,j) = 1$ (i.e., never use toll roads in W'). For simplicity, assume W' is pre-computed for you. For $0 \le k \le n$, let

- D^k be the matrix of all shortest distances w.r.t. W' which only use nodes $\le k$ as intermediate nodes.
- C^k be the matrix or all shortest distances w.r.t. W which only use nodes $\le k$ as intermediate nodes, but *also use at most one toll*.

Fill in the blanks below to directly modify the Floyd-Warshall algorithm to compute the correct answer for problem (c) in time $O(n^3)$. Argue the correctness of your algorithm.

```
FLOYD-WARSHALL(W, W')
    n = W.rows
    D^0 = ......
    C^0 = ......
    for k = 1 to n do
        C^k = (c^k_{i,j}) be a new n × n matrix
        D^k = (d^k_{i,j}) be a new n × n matrix
        for i = 1 to n do
            for j = 1 to n do
                d^k_{i,j} = min (.....................................................)
                c^k_{i,j} = min (.....................................................)
    return .............
```

Solution:

FLOYD-WARSHALL(W, W')

$n = W.rows$

$D^0 = W'$

$C^0 = W$

for $k = 1$ **to** n **do**

$\quad C^k = (c^k_{i,j})$ be a new $n \times n$ matrix

$\quad D^k = (d^k_{i,j})$ be a new $n \times n$ matrix

$\quad$ **for** $i = 1$ **to** n **do**

$\quad\quad$ **for** $j = 1$ **to** n **do**

$$d^k_{i,j} = \min\left(d^{k-1}_{i,k} + d^{k-1}_{k,j}, d^{k-1}_{i,j}\right)$$

$$c^k_{i,j} = \min\left(d^{k-1}_{i,k} + c^{k-1}_{k,j}, c^{k-1}_{i,k} + d^{k-1}_{k,j}, c^{k-1}_{i,j}\right)$$

return D^n

$\square$